



南開大學
Nankai University

计算机学院
并行程序设计实验报告

GPU 编程

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2025 年 7 月 2 日

目录

1 实验目标	3
2 实验平台配置	3
2.1 硬件配置	3
2.2 软件环境	3
3 算法设计与实现	3
3.1 GPU 基础加速版本	3
3.1.1 核心设计思路	3
3.1.2 关键实现策略	4
3.1.3 关键代码	4
3.2 GPU Barrett 规约版本	4
3.2.1 Barrett 规约原理	5
3.2.2 GPU 优化策略	5
3.2.3 关键代码	5
3.3 GPU Montgomery 规约版本	5
3.3.1 Montgomery 规约原理	5
3.3.2 关键设计要点	6
3.3.3 关键代码	6
4 实验结果分析	6
4.1 测试数据集说明	6
4.2 性能测试结果	7
4.3 时间复杂度分析	7
4.3.1 理论时间复杂度	7
4.3.2 GPU 并行化效果分析	7
4.4 加速比分析	7
4.4.1 数据集 0 分析（小规模）	7
4.4.2 数据集 1-3 分析（中大规模）	7
4.4.3 规约优化效果分析	8
5 性能分析与讨论	8
5.1 GPU 加速效果评估	8
5.1.1 正向加速效果	8
5.1.2 规约算法对比	8
5.2 性能瓶颈分析	8
5.2.1 GPU 架构限制	8
5.2.2 算法特性影响	8
5.3 规模敏感性分析	9
5.3.1 小规模问题 ($n < 1000$)	9
5.3.2 中大规模问题 ($n \geq 100000$)	9

6	实验结论	9
7	链接	9

1 实验目标

本实验旨在设计并实现基于 CUDA 的数论变换 (NTT) 多项式乘法并行算法，通过 GPU 并行计算技术加速大规模多项式乘法运算。具体目标包括：

1. 实现 GPU 基础版本的 NTT 算法，探索 GPU 并行计算在数论变换中的应用潜力
2. 设计并实现 GPU Barrett 规约版本，优化模运算性能，减少计算开销
3. 设计并实现 GPU Montgomery 规约版本，进一步提升模运算效率
4. 对比分析五种不同实现方式的性能表现，量化 GPU 加速效果
5. 深入分析不同规约算法在 GPU 架构下的适用性和性能特征

2 实验平台配置

2.1 硬件配置

实验使用的 GPU 硬件配置如下：

- GPU 型号：GRID T4-8Q
- SM 数量：40 个流式多处理器
- 最大线程数/块：1024
- 最大网格大小：2,147,483,647

2.2 软件环境

- 编程语言：C++/CUDA
- 编译器：NVCC
- 运行时：CUDA Runtime API

3 算法设计与实现

3.1 GPU 基础加速版本

GPU 基础版本采用直接的 CUDA 并行化策略，将传统 CPU 串行 NTT 算法转换为 GPU 并行实现。

3.1.1 核心设计思路

1. 位逆序置换并行化：这是一个典型的“易并行”任务。每个元素的目标位置可以独立计算，不受其他元素影响。因此，可以启动一个 CUDA 核函数，让每个线程负责一个数组元素的位逆序交换。
2. 蝶形运算并行化：在每个 NTT 层级中，不同蝶形运算组之间相互独立，可并行执行

3. 内存管理优化：采用统一内存管理，减少 CPU-GPU 数据传输开销
4. 点值乘法与最终缩放：这两步也是易并行的。每个元素的计算都是独立的。因此，可以启动一个核函数，让每个线程负责数组中一个位置的乘法运算。
5. 模运算：在基础版本中，我们直接使用 CUDA C++ 中的 ‘%’ 运算符进行模乘和模加减运算。这虽然简单，但 ‘%’ 操作 (尤其是对 64 位整数) 在硬件层面通常会编译成较慢的除法指令，是潜在的性能瓶颈。

3.1.2 关键实现策略

核心蝶形运算采用以下并行策略：

- 外层循环遍历不同长度的蝶形运算 ($h = 2, 4, 8, \dots$)
- 中层使用 grid 并行化不同的蝶形运算组
- 内层使用 block 并行化同组内的蝶形运算对
- 预计算并存储旋转因子，避免重复计算

3.1.3 关键代码

蝶形运算是 NTT 的核心，其基础 GPU 核函数 `ntt_basic_kernel` 的设计如下。该核函数由 `len/h` 个 Block 启动，每个 Block 处理一个大小为 `h` 的蝶形组。

Listing 1: 基础 NTT 蝶形运算核函数 `ntt_basic_kernel`

```

1  __global__ void ntt_basic_kernel(LL *a, int len, int h, LL *w_table, LL p) {
2      // blockIdx.x 标识当前是第几个蝶形运算组
3      int j_base = blockIdx.x * h;
4      // threadIdx.x 负责组内的计算
5      for (int k = threadIdx.x; k < h / 2; k += blockDim.x) {
6          int idx1 = j_base + k;
7          int idx2 = j_base + k + h / 2;
8          LL w = w_table[k]; // 从预计算表中获取旋转因子
9          // 核心蝶形运算，使用128位整数避免中间结果溢出
10         unsigned __int128 product = (unsigned __int128)a[idx2] * w;
11         LL t = product % p;
12         a[idx2] = (a[idx1] - t + p) % p;
13         a[idx1] = (a[idx1] + t) % p;
14     }
15 }
```

3.2 GPU Barrett 规约版本

Barrett 规约是一种高效的模运算优化技术，特别适用于固定模数的重复模运算场景。

3.2.1 Barrett 规约原理

Barrett 规约通过预计算模数的倒数近似值，将除法运算转换为乘法和位移运算：

$$x \bmod p \approx x - \lfloor \frac{x \cdot \mu}{2^{64}} \rfloor \cdot p$$

其中 $\mu = \lfloor \frac{2^{64}}{p} \rfloor$ 为预计算的倒数近似值。

3.2.2 GPU 优化策略

1. 预计算优化：在 CPU 端预计算 Barrett 参数，避免 GPU 端重复计算
2. 内存访问优化：将 Barrett 参数存储在 GPU 常量内存中，提高访问效率
3. 精度控制：使用 128 位中间计算保证精度，最终结果转换为 64 位
4. 溢出处理：增加边界检查，处理可能的下溢情况

Barrett 规约将模运算的复杂度从 $O(\log p)$ 降低到 $O(1)$ ，显著提升了 GPU 并行效率。

3.2.3 关键代码

Barrett 规约的核心在于其设备端乘法函数，它利用 128 位整数和预计算的 ‘im’ 来避免除法。

Listing 2: GPU Barrett 模乘设备函数 `gpu_barrett_multiply`

```

1  struct GPUBarrett { ULL mod, im; };
2
3  __device__ ULL gpu_barrett_multiply(ULL a, ULL b, const GPUBarrett* barrett) {
4      unsigned __int128 z = (unsigned __int128)a * b;
5      // 关键步骤：将除法替换为乘法和移位
6      unsigned __int128 x = ((unsigned __int128)z * barrett->im) >> 64;
7      unsigned __int128 y = x * barrett->mod;
8      ULL v = (ULL)(z - y);
9      // 处理下溢和边界情况
10     if (z < y) v += barrett->mod;
11     if (v >= barrett->mod) v -= barrett->mod;
12     return v;
13 }
```

3.3 GPU Montgomery 规约版本

Montgomery 规约是另一种高效的模运算技术，通过域转换避免传统除法运算。

3.3.1 Montgomery 规约原理

Montgomery 规约工作在 Montgomery 域中，将模运算转换为：

$$\text{MonRed}(aR, bR) = \frac{(aR)(bR)}{R} \bmod p = abR \bmod p$$

其中 $R = 2^{64}$ 为 Montgomery 基数。

3.3.2 关键设计要点

1. 域转换管理:

- 输入数据转换到 Montgomery 域: $a \rightarrow aR \bmod p$
- NTT 计算在 Montgomery 域内进行
- 输出结果转换回普通域: $aR \rightarrow a \bmod p$

2. 参数预计算:

- 计算 $R^2 \bmod p$ 用于域转换
- 计算 p' 满足 $pp' \equiv -1 \pmod{2^{64}}$

3. GPU 内存优化: 所有 Montgomery 参数统一管理, 减少内存访问开销

Montgomery 规约的优势在于避免了除法运算, 所有操作都是乘法和位运算, 更适合 GPU 并行架构。

3.3.3 关键代码

Montgomery 实现的关键是其域内乘法函数和域转换函数。以下是域内乘法的设备函数。

Listing 3: GPU Montgomery 模乘设备函数 `montgomery_reduce`

```

1  struct MontgomeryParams { ULL mod, n_prime, r_squared; };
2
3  // T = a * b, a和b已在Montgomery域
4  __device__ ULL montgomery_reduce(unsigned __int128 T, const MontgomeryParams*
   params) {
5      // 关键步骤: 同样避免了显式的大数除法
6      ULL m = (ULL)T * params->n_prime;
7      unsigned __int128 t = (T + (unsigned __int128)m * params->mod) >> 64;
8      return (t >= params->mod) ? (t - params->mod) : t;
9  }
10
11 __device__ ULL montgomery_multiply(ULL a, ULL b, const MontgomeryParams* params) {
12     return montgomery_reduce(((unsigned __int128)a * b, params);
13 }

```

4 实验结果分析

4.1 测试数据集说明

实验使用 4 个不同规模和模数的测试数据集:

- 数据集 0: $n = 4, p = 7340033$ (小规模测试)
- 数据集 1: $n = 131072, p = 7340033$ (中等规模, 小模数)
- 数据集 2: $n = 131072, p = 104857601$ (中等规模, 中等模数)
- 数据集 3: $n = 131072, p = 469762049$ (中等规模, 较大模数)

4.2 性能测试结果

表 1: 不同版本在各测试集上的执行时间 (单位: ms)

实现版本	测试集 0	测试集 1	测试集 2	测试集 3
	n=4 p=7340033	n=131072 p=7340033	n=131072 p=104857601	n=131072 p=469762049
CPU Barrett 规约	0.011	61.502	61.710	61.815
CPU Montgomery 规约	0.005	48.606	48.801	48.537
GPU 基础版本	124.330	24.406	26.197	24.817
GPU Barrett 规约	2.424	18.537	17.851	18.183
GPU Montgomery 规约	0.714	19.902	16.790	18.114

4.3 时间复杂度分析

4.3.1 理论时间复杂度

对于长度为 n 的多项式乘法:

- 传统方法: $O(n^2)$
- NTT 方法: $O(n \log n)$

4.3.2 GPU 并行化效果分析

GPU 并行化主要优化以下操作:

1. 位逆序置换: 从 $O(n)$ 串行降低到 $O(1)$ 并行
2. 蝶形运算: 每层 $O(n)$ 运算并行化
3. 点值乘法: $O(n)$ 完全并行化

4.4 加速比分析

4.4.1 数据集 0 分析 (小规模)

在小规模数据 ($n = 4$) 下:

- GPU 基础版本相对 CPU 版本出现**负加速**现象
- 原因: GPU 启动开销 (kernel launch overhead) 远大于实际计算时间
- GPU 初始化和内存传输成本在小规模下占主导地位

4.4.2 数据集 1-3 分析 (中大规模)

在中大规模数据 ($n = 131072$) 下:

表 2: GPU 相对 CPU 加速比

对比项	数据集 1	数据集 2	数据集 3
GPU 基础 vs CPU Barrett	2.52×	2.36×	2.49×
GPU 基础 vs CPU Montgomery	1.99×	1.86×	1.96×
GPU Barrett vs CPU Barrett	3.32×	3.46×	3.40×
GPU Montgomery vs CPU Montgomery	2.44×	2.91×	2.68×

4.4.3 规约优化效果分析

表 3: GPU 内部不同版本加速比

优化效果	数据集 1	数据集 2	数据集 3
Barrett vs 基础版本	1.32×	1.47×	1.36×
Montgomery vs 基础版本	1.23×	1.56×	1.37×

5 性能分析与讨论

5.1 GPU 加速效果评估

5.1.1 正向加速效果

在合适规模下 ($n \geq 131072$), GPU 版本展现出明显的加速效果:

Barrett 规约版本表现最佳, 相对 CPU 实现达到 $3.32\times$ - $3.46\times$ 的加速比; 基础 GPU 版本也达到 $1.86\times$ - $2.52\times$ 的加速比; Montgomery 规约版本达到 $2.44\times$ - $2.91\times$ 的加速比。

5.1.2 规约算法对比

- Barrett 规约:
 - 优势: 预计算开销小, 适合 GPU 并行
 - 在 GPU 上相对基础版本提升 32%-47%
- Montgomery 规约:
 - 优势: 避免除法运算, 计算效率高
 - 域转换开销在大规模下被摊销
 - 在 GPU 上相对基础版本提升 23%-56%

5.2 性能瓶颈分析

5.2.1 GPU 架构限制

1. 内存带宽限制: NTT 算法存在大量不规则内存访问
2. 线程分歧: 条件分支导致 warp 内线程执行不一致
3. 同步开销: 不同 NTT 层级间需要全局同步

5.2.2 算法特性影响

1. 数据依赖: NTT 的蝶形运算存在层级间数据依赖
2. 访问模式: 位逆序和蝶形运算的内存访问模式不规则
3. 计算密度: 相对于内存访问, 计算量较小

5.3 规模敏感性分析

GPU 加速效果与问题规模密切相关：

5.3.1 小规模问题 ($n < 1000$)

- GPU 启动开销占主导，出现负加速
- 建议使用 CPU 版本

5.3.2 中大规模问题 ($n \geq 100000$)

- GPU 并行优势明显，加速比稳定在 2-3 倍
- Barrett 规约版本表现最佳
- 建议优先使用 GPU Barrett 规约版本

6 实验结论

本实验成功实现了基于 CUDA 的 NTT 多项式乘法并行加速，主要结论如下：

1. **GPU 加速有效性：**在合适规模下 ($n \geq 131072$)，GPU 版本相对 CPU 版本可达到 2-3.5 倍加速比
2. **规约算法优势：**Barrett 和 Montgomery 规约在 GPU 上都能带来额外的 30%-50% 性能提升
3. **Barrett 规约最优：**在本实验环境下，GPU Barrett 规约版本表现最佳
4. **规模敏感性：**GPU 加速效果强烈依赖于问题规模，小规模问题不适合 GPU 加速

通过本实验，验证了 GPU 并行计算在数论变换优化中的显著效果，为大规模多项式运算提供了高效的解决方案。

7 链接

Gitee 链接 [Gitee](#)